

Project Report

Design and Verification of an 8-bit Sequential Multiplier

Reece M. Dobro

Abstract

The process of designing and testing an 8-bit by 8-bit unsigned sequential multiplier using the Verilog hardware description language within AMD Vivado is presented in this report. In order to evaluate and examine the module, a simulation was created and shown in this report. The results of the simulation were analyzed using waveform outputs and TCL console logs, which confirmed that the outputs of the module matched the expected values. In addition, the RTL Schematic from AMD Vivado was examined and compared to the digital design of the modules. Overall, the simulation results demonstrate that the design operates correctly and satisfies the functional requirements specified by the design specifications.

1 Introduction, Background, and Theory

The main objective of this project is to design, program, and test an 8-bit by 8-bit unsigned sequential multiplier. Due to the use of sequential logic, a 1-bit clock input is required to control state updates. The multiplication operation is performed similarly to handwritten multiplication through the use of the “shift-and-add” method. For this method, the multiplicand (the value being multiplied) is added to an accumulated result depending on the value of the LSB (least significant bit) of the multiplier (the value by which the multiplicand is multiplied). If the LSB of the multiplier is 1, then the addition occurs; otherwise, the addition does not occur. After each LSB evaluation, the multiplicand is shifted left, and the multiplier is shifted right, and the process is repeated until all bits have been evaluated. The final product is held at the output of the design until the 1-bit input indicator, *start*, is driven high. By design specifications, when *start* is driven high, the operands are processed by the design.

1.1 Inputs and Outputs

The inputs, outputs, and expected behavior were provided by the design specifications for this system. The inputs and outputs for the multiplier module are shown in Table 1.

Signal	Direction	Size (bits)	Description
multiplicand	Input	8	The value that is multiplied (Unsigned)
multiplier	Input	8	The value that multiplies the multiplicand (Unsigned)
start	Input	1	Indicator to begin multiplying the inputs given.
clk	Input	1	Clock signal due to sequential logic.
product	Output	16	The product of the two 8-bit inputs .

Table 1: Port Signal and Function Table for the 8 by 8 Multiplier

1.2 Design of 8 by 8 Multiplier

Figure 1 shows the “Pen and Paper” design of the 8 by 8 multiplier. The module name for the multiplier is *mult_seq_8x8*, as per design specifications. As shown in Figure 1, the values of both 8-bit operands are stored in separate registers, and the 16-bit product is stored in its own register. This is done due to the sequential logic required by design specifications. Due to the 2-to-1 multiplexers driving the inputs to the registers, when *start* is driven high, the *multiplier* and *multiplicand* registers are assigned the values at their respective inputs. The product register, when *start* is driven high, is assigned a value of 16'd0, as per design specifications. When *start* is driven low, the input to the multiplier register is the previous output logically shifted right by 1-bit. The input to the multiplicand register, however, is the

previous output logically shifted left by 1 bit. While the input size of *multiplicand* is 8 bits, internally, it is 16-bits due to its value being shifted left eight times. As *multiplicand* is 16-bit wide internally, it is assigned to a 16-bit register. Regarding the product register, which contains the partial product, its value is dependent on the value of the multiplier's LSB. If the LSB is 1'b1, then the value of the product is added to the value of the multiplicand. If the LSB is 1'b0, then the value of the partial product remains the same. This behavior is performed to ensure correct accumulation during the shift-and-add multiplication process. This design, after eight cycles, has the product register containing the correct product of the multiplier and multiplicand. Even after the 8th cycle, the value of *multiplier* is 8'd0 and remains 8'd0 due to being repeatedly logically shifted right. This results in the value of the LSB of the *multiplier* always equal to 1'b0, causing the value of *product* to remain the same.

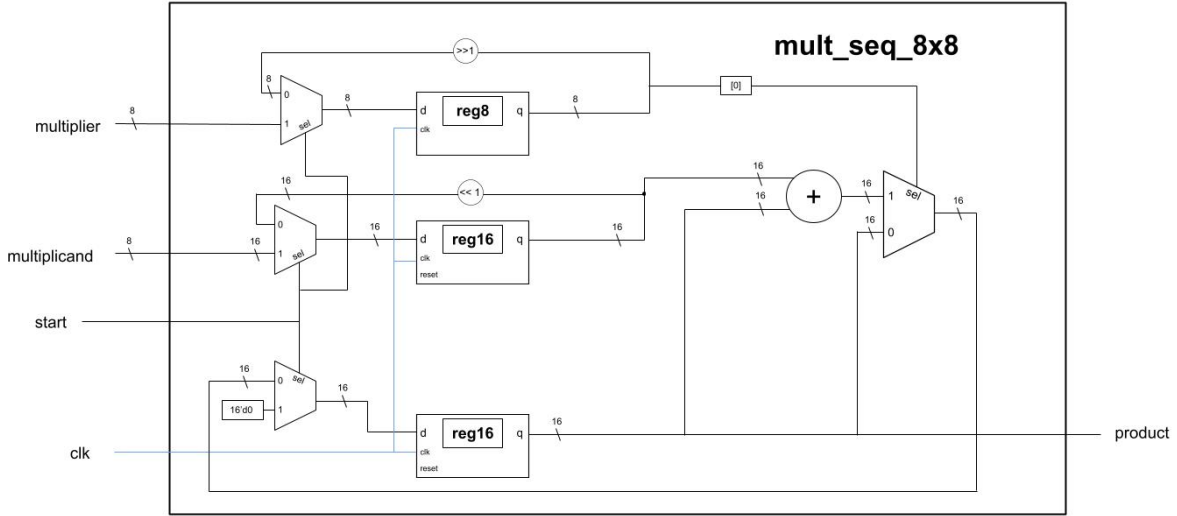


Figure 1: "Pen and Paper" Design of the 8 by 8 Multiplier

1.3 Verilog Implementation

```

module mult_seq_8x8(
    input [7:0] multiplier,
    input [7:0] multiplicand,
    input start,
    input clk,
    output reg [15:0] product
);
    reg [7:0] multi;
    reg [15:0] mcand;
    always @(posedge clk) begin
        if (start) begin
            multi <= multiplier;
            mcand <= {8'd0, multiplicand};
            product <= 16'd0;
        end
        else begin
            if (multi[0]) product <= product + mcand;
            else product <= product;
            multi <= multi >> 1;
            mcand <= mcand << 1;
        end
    end
endmodule

```

Figure 2: Source Code of Module *mult_seq_8x8*

The Verilog implementation of the 8 by 8 bit sequential multiplier, *mult_seq_8x8*, is shown in Figure 2. While the module is written at the behavioral level, it could have been written at the dataflow level and contain instances of register modules. This was not done, however, as design specifications required that all code be written at the behavioral level. The source code of the module is based on the “Pen and Paper” design from Figure 1. As this is written at the behavioral level, the 16-bit output, *product*, is declared as a register data type. The Verilog source code for the module also contains an 8-bit net, *multi*, and a 16-bit net, *mcand*. The 16-bit net, *mcand*, is used to transform the 8-bit input, *multiplicand*, into an equivalent 16-bit value. As the input operands are unsigned, *multiplicand* will always be sign extended with zero. The 8-bit net, *multi*, serves as a working copy of the multiplier used to control the shift-and-add process. The 8-bit input port, *multiplier*, can’t be used as a mutable version is needed that changes every cycle.

2 Simulation, Results and Analysis

To verify the accuracy of the *mult_seq_8x8* module, a simulation through AMD Vivado will be executed. The simulation will consist of two sets of 8-bit operands being entered as inputs, with the outputs of the module and its behavior being analyzed to determine if it is successful. In addition to outputting the correct value after eight cycles, the module is expected to output the correct partial sum through each stage. The test values chosen for the simulation are shown in Table 2.

Test Case	Multiplier [7:0]	Multiplicand [7:0]	Expected Product [15:0]
1	8'b11111111	8'b00000010	16'd510
2	8'b10101010	8'b10001001	16'd23290

Table 2: Sequential 8x8 Multiplier *mult_seq_8x8* Test Vectors

The first set of test vectors listed in Table 2 is a boundary test case. Its primary purpose is to verify correct operation at the maximum 8-bit unsigned operand value (8'd255) and ensure proper carry propagation from the adder into the upper bits of the product register. By using a maximum-value operand, this test case ensures that the design correctly utilizes the full 16-bit product register and does not overflow the partial products. This confirms that the shift-and-add logic properly propagates carries across all stages of the multiplication process. In addition, it is verified that the sequential control logic properly updates the product register without loss of partial product data during shifting operations. This is achieved by having the multiplier LSB be high for the entire duration of the multiplication process.

The second set of test vectors listed in Table 2 is a distributed bit test case and a large value test case. The first purpose is to verify that the shift-and-add logic works correctly even when non-consecutive additions of the partial product and the multiplicand are performed. For the test case, this is achieved by having the multiplier’s LSB alternating between high and low over the course of the shift and add process. The second purpose is to verify that the shift-and-add logic works correctly even with a large value for the two operands. For the test case, this is achieved by having the MSB of both operands be 1'b1.

To execute the test cases, one testbench was created in Verilog. The simulation will output its critical information through both the TCL Console and waveforms. The waveforms output ensures that ports are mapped according to design specifications and can help to debug any issues discovered during testing. The TCL Console displays outputs in a text-based format that allows for simpler analysis of the ports. The inputs for each simulation match those given in Table 2. The list below contains the conditions for the simulation.

- The time unit of the simulation will be 1 ns.
- The period of the 1-bit input *clock* will be 10 ns.
- After 8 clock cycles from the negative edge of *start*, the output will be measured
- The delay between the start of the first set of test vectors and the second set will be 120 ns

2.1 Simulation Results

Table 3 contains the results of the final product outputs of the simulation compared to the expected results. The TCL Console output from the simulations was compiled into the tables to show the actual outputs and allow for direct comparison to the expected outputs. As shown by Table 3, the outputs

Test Case	Inputs		Expected Output	Actual Output
	Multiplier [7:0]	Multiplicand [7:0]	Product [15:0]	Product [15:0]
1	8'b11111111	8'b00000010	16'd510	16'd510
2	8'b10101010	8'b10001001	16'd23290	16'd23290

Table 3: Sequential 8x8 Multiplier *mult_seq_8x8* Test Vector Results

of the multiplier module exactly match the expected outputs.

The waveforms output of the first test case is shown in Figure 3. For the simulation, the first test case occurs from 0 ns to 120 ns. The radix for all values is decimal to match the formatting of Table 3. As shown by the figure, eight cycles after the test vectors were given and the *start* bit was toggled, the output of the module was *product* = 16'd510. From 0 ns to 6 ns, the *start* input bit was high, to allow the inputs to be processed. 80 ns after the *start* was driven low (85 ns), the value of *product* with the full and correct product of the two operands is given. This demonstrates the correct timing logic of the multiplier as the clock has a period of 10 ns, resulting in the correct output being given after 8 cycles of the clock. This matches the expected behavior given by the design specifications. Furthermore, between the start of the simulation and *product* holding the full correct product, *product* increases. These increases are due to the value derived from the multiplicand being added to the partial product. The values of the product throughout the first simulation match the expected partial products of the multiplier and multiplicand as they shift. This demonstrates that the multiplier is utilizing the shift and add method in order to compute the final product, matching another behavior set by design specifications. Additionally, after 85ns, when the full and correct product value is the output for *product*, the value is held until the next set of input values are applied for the second test case, at 120 ns. This confirms another expected behavior defined in the design specifications.

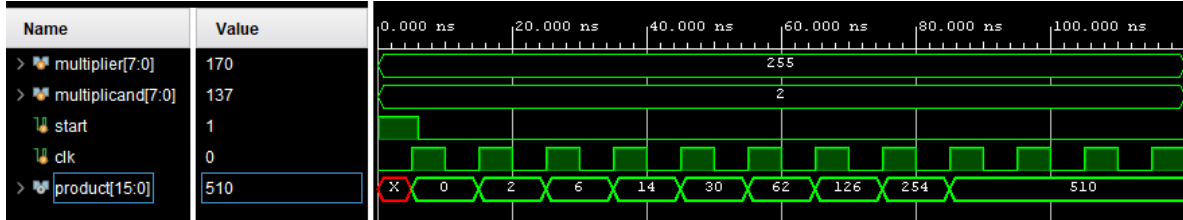


Figure 3: Waveforms Output of First Test Case

The waveforms output of the second test case is shown in Figure 4. For the simulation, the second test case occurs from 120 ns to 240 ns. The radix for all values is decimal to match the formatting of Table 3. As shown by the figure, eight cycles after the test vectors were given and the *start* bit was toggled, the output of the module, *product*, was 16'd23290. From 120 ns to 126 ns, the *start* input bit was high, to allow the inputs to be processed. During the duration, the value of *product* became 16'd0, matching the appropriate behavior set in the "pen and paper" design and design specifications. 80 ns after the *start* was driven low (205 ns), the value of *product* with the full and correct product of the two operands occurs. This demonstrates the correct timing logic of the multiplier due to the same reasons as for the previous test cases. Furthermore, between the start of the simulation and *product* holding the full correct product, *product* increases. These increases occur for the same reason as the previous test case, partial product accumulation. As these increases occur and match the expected partial products for the operands, it is shown that the multiplier is utilizing the shift and add method in order to compute the final product. Additionally, after 205ns, when the full and correct product value is the output for *product*, the value is held until the end of the simulation, at 240 ns. The value being held occurs due to the shifting logic and partial product logic shown in the "Pen and Paper" design.

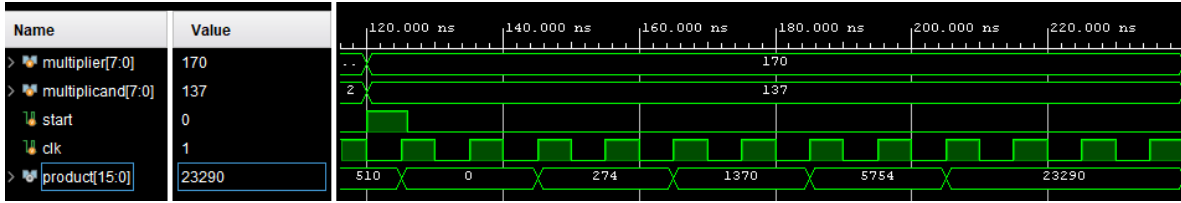


Figure 4: Waveforms Output of Second Test Case

Figure 5 provides an expanded view of the output waveform between 0 ns and 8 ns. The figure shows the output, *product*, initially having an unknown value, 16'bx. This behavior is also observed in Figure 3, although less clearly. As shown in 5, the output changes to a known value, 16'd0, at 5 ns. This occurs due to the outputs *product*, having registers to hold its value. When a register is first initialized in Verilog, it has a value of X, which is why the outputs begin the simulation with a value of X. The registers for this module, as per design specifications, update on the positive edge of the clock signal. As the clock signal's positive edge occurs at 5 ns into the simulation, from 0 ns to 5 ns, the value of the registers that drive the outputs is unknown before 5 ns. In order to have the outputs be defined for the entirety of the simulation, the clock signal would need to begin high, not low. This initial delay is an inherent characteristic of register logic rather than a functional error. From this analysis, the initial unknown state of the output is expected behavior rather than a result of a functional error in the design.

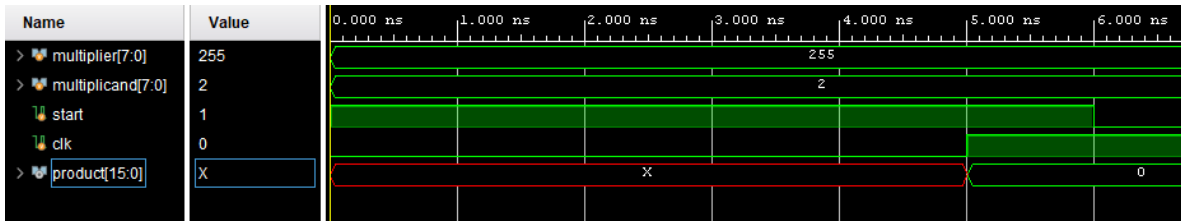


Figure 5: Enhanced Waveforms Output from 0 ns to 8 ns

As the simulation's outputs match the expected outputs and the waveforms show the correct behavior of the multiplier, the module, and the "Pen and Paper" design are deemed successful.

A potential limitation in the verification process is that only a limited number of test cases were utilized. For the 8-bit by 8-bit unsigned sequential multiplier, with two 8-bit inputs, there are $2^8 \times 2^8 = 2^{16} = 65,536$ possible input combinations. While the simulation scenarios cover the primary operations and edge cases, they represent only a fraction of the total input space. However, it is impractical to simulate every possible 8-bit sequence due to the time required for manual verification and documentation. This is why representative test cases, specifically boundary cases and multi-digit ripple carry sequences, were selected to confirm the design's success under a variety of logical conditions.

Both modules are deemed successful as the output waveforms and TCL binary logs match the expected behavior set by the design specifications and match the expected outputs.

2.2 RTL Schematic Results and Comparison

Figure 6 shows the schematic of the RTL Analysis from AMD Vivado for the multiplier module. These figures provide a view of the synthesized design as interpreted by the AMD Vivado. The Verilog program written in AMD Vivado is translated into basic hardware components such as multiplexers and logic gates. It is important to note that the single net exiting the top of the figure is the clock signal.

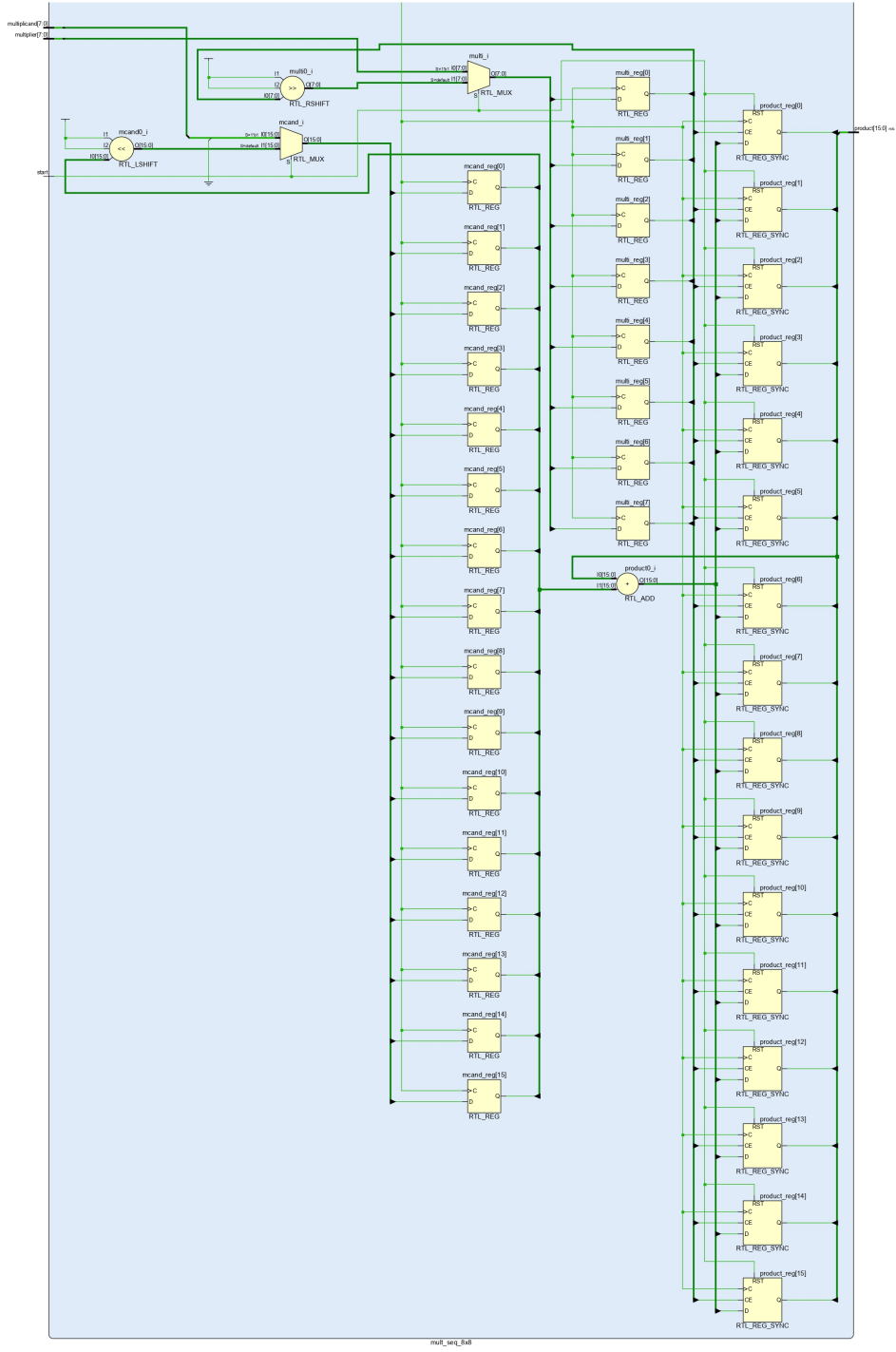


Figure 6: RTL Schematic result for the module, *mult_seq-8x8*

The schematic of the 8 by 8 multiplier, Figure 6, matches closely with the expected results from the “Pen and Paper” diagram created for the module, Figure 1. The main difference between the two diagrams is the representation of the three registers, the 8-bit used for *multiplier* and the two 16-bit registers used for *multiplicand* and *product*. In the RTL schematic, the three registers are shown as a collection of D Flip Flop blocks, with the number of D Flip Flops for each register matching the size of the registers shown in the “Pen and Paper” design. The collection of D Flip Flops are grouped

together in the "Pen and Paper" design to form register blocks. A reset port is used for the D Flip Flops in the RTL schematic, while a 2:1 multiplexer is used for the "Pen and Paper" design. Another difference between the two designs are the shifting blocks used for the *multiplier* and *multiplicand*. To show a shift by 1-bit, the RTL Analysis has a connection to high for the shift block. The number 1 and shift operation (`||1` and `||1`), as it would look in Verilog, is used for the "Pen and Paper" design. The final major difference between the two designs is the implementation of the *multiplier* LSB logic for the partial product. A clock enable bit is used for the D Flip Flops that make up the *product* in the RTL Analysis. A 2-to-1 multiplexer is used instead in the "Pen and Paper" design. Overall, the major elements remain the same for both designs, along with the connections between these elements. Due to the close similarity between these designs, it can be concluded that the Verilog implementation of the "Pen and Paper" diagram was successfully done.

3 Conclusions

In this project, the correct design, implementation, and verification of an 8-bit by 8-bit unsigned sequential multiplier was achieved. The simulation results confirmed that the module functions appropriately and was able to use registers and shifting operations to find the product of two 8-bit numbers. Simulation results also confirmed that the module can handle multiplication under both typical operation and edge cases. The implementation was shown to be successful, with the RTL schematic matching the digital design. Future work could include resource analysis to identify potential hardware modifications that would improve overall efficiency. Overall, this project demonstrates how Verilog can be used to implement and verify arithmetic modules digitally, producing modular designs without requiring physical components or additional hardware.

This report was compiled using L^AT_EX .